From deliveries to atom energy

A detailed judging-preparation guide | 24 October 2026 | Repository snapshot d7027f7

Pasqal CVRP hackathon: Track A, Track B, mathematical derivations, physical controls, saved experiments, and an honest account of what is implemented.

This guide is for a team member who wants to explain every link from a delivery decision to a measured atom bitstring. It starts with everyday meanings, derives the equations, and connects them to the current Python functions. It is a study companion, not the four-page competition submission.

The direct answers

Did we use QUBO? Yes. Track A samples a quadratic binary subset-selection energy; Track B implements two explicit TSP QUBOs. The textbook full assignment-and-slack CVRP construction is derived for comparison, but is not the implemented quantum capacity solver.

Did we run quantum hardware? No. The measured samples are generated by Pulser's QutipBackendV2 on a classical CPU. The algorithms are quantum-assisted emulator experiments with substantial classical computation.

What must not be confused? Actual travel distance, the grouping surrogate, target binary energy, and realized physical energy. They are connected by modeling choices and approximations, not identical by definition.

What was checked for this guide

Missing GitHub updates were fast-forwarded into the local project. All 11 repository tests passed; pip check found no broken requirements. Small-example references and energies were recomputed for explanation. Complete official benchmarks, the US timing repeats, and the space experiment were not regenerated. Their numerical results below are identified as saved evidence.

Main source: ericmao2007/DuQuantum-Pasqal, commit d7027f7f698c03bb9aeae287a796f1742b233ee6. Code and team write-ups are evidence of implementation, not an external peer review. Source key and exact file map appear at the end.

How to study this guide

Reading map and the central chain

Pages	Question answered
3-5	What problem are we solving, and what do the symbols mean?
6-9	What is a QUBO, and what is Track A actually minimizing?
10-15	How do energy, atom positions, detuning, and pulses fit together?
16-18	Can we calculate a real four-customer example?
19-22	How does Track B encode and measure a route?
23-26	How are references, scores, plots, and timing interpreted?
27-29	What do the demos and code do, and what remains limited?
30-34	Glossary, judge questions, source trail, and reproduction
35-37	Expanded penalties, Ising physics, and the routing comparison plot

The chain you should be able to draw

Delivery requirements → constrained model → binary variables → target QUBO → physical embedding → pulse evolution → measured bitstrings → validity checks → delivery routes → evaluation.

An error at an early link is not cured by a later link. A perfect pulse cannot correct an inappropriate grouping objective. A valid mathematical QUBO does not imply that a planar atom array reproduces it. A plausible route picture does not prove optimality.

Four levels of cost

Quantity	Meaning	Units
C	Total warehouse-to-customers-and-back travel	Original distance units
F	Compactness plus group-size balancing	Dimensionless
E_target	Binary energy proposed to the device	Dimensionless or scaled angular frequency; label it
H / hbar	Actual simulated quantum generator	rad per microsecond

Say it to the judges: We use a compact sequential heuristic. The quantum component proposes choices; classical code validates them and evaluates the resulting routes. We separately measure the model-to-physics mismatch.

The original delivery problem

[R1] Challenge brief, sections 2, 5, 6; [R2] core.py and MODEL.md

CVRP means Capacitated Vehicle Routing Problem. Customers have locations and integer demands; identical vehicles have a common capacity Q. The depot is the fixed warehouse. We want every customer served once, no overloaded vehicle, and a short total return-trip distance.

Symbol	Definition
N / K / Q	Number of customers / number of vehicles / capacity per vehicle
p_i, d_i	Location and demand of customer i
c_ij	Distance from location i to j; depot has index 0 in the formal equations
y_ik	1 if vehicle k serves customer i, otherwise 0
z_ijk	1 if vehicle k travels directly from i to j
C*	Best possible feasible total distance

$$c_{ij} = \|p_i - p_j\|_2, \qquad C = \sum_{k}\sum_{i \neq j} c_{ij} z_{ijk}$$

Euclidean distance is the straight-line ruler distance. It is symmetric, c_ij = c_ji, and satisfies the triangle inequality. We are not using roads, traffic, time windows, fuel consumption, or repeated warehouse reloads.

A vehicle may be unused. A used vehicle departs the depot once and returns once. Deliveries cannot be split between vehicles. Total demand below total fleet capacity is necessary but not sufficient for feasible packing: three demands of 2 cannot fit in two vehicles of capacity 3.

Formal notation versus Python indices

Equations label the depot 0 and customers 1 through N. Python stores customers as 0 through N-1 and the depot separately. Functions temporarily shift customer indices when building distance arrays. A displayed customer 0 is a customer, not the warehouse.

Say it to the judges: Our original objective is total distance. The grouping score is a surrogate that makes a smaller quantum subproblem possible.

Constraints: what makes an answer legal?

[R1] Challenge assumptions; [R2] formal CVRP and core.validate

$$\sum_k y_{ik} = 1, \qquad \sum_i d_i y_{ik} \le Q$$

The first rule is an **equality**: exactly one vehicle serves each customer. The second is an **inequality**: a vehicle can carry anything from zero up to capacity. Underfilling is allowed.

$$\sum_{j \neq i} z_{ijk} = y_{ik}, \qquad \sum_{j \neq i} z_{jik} = y_{ik}$$

These degree equations require one arrival and departure at each served customer. Depot departures equal depot arrivals and are at most one per vehicle. Degree rules alone can allow isolated cycles; subtour constraints rule them out.

$$\sum_{i,j \in S,\ i \neq j} z_{ijk} \le |S| - 1$$

For every nonempty customer subset S, the last inequality prevents a complete closed loop isolated from the depot. The full formal model describes CVRP; the implementation does not assign an atom to every y and z variable.

Validation and certificates

A feasibility certificate is a known legal customer-to-vehicle assignment shipped with the instance. It proves at least one plan exists. The sampler does not receive the certificate as a shortcut. validate checks assignment length and labels, demand length and integer types, loads, finite coordinates and bounds, allowed demand ranges, and tier dimensions.

Earlier code accepted a truncated demand array because zip silently stopped early. The current pulled code explicitly checks demand length and finite coordinate shape. That particular historical finding is fixed upstream; tests still should be expanded for malformed inputs beyond the current fixtures.

Say it to the judges: A cheap answer that overloads a vehicle is not a valid solution. We report failure, not a misleading low gap.

Instances and the two-stage approximation

[R2] core.generate, generate_heterogeneous, clustering_cost; [R1] tier table

Tier	N	K	Q	Allowed demand	Uniform demand
S	4	2	7	1-4	3
M	10	3	20	1-8	5
L	15	4	30	1-10	6

Official examples use a 100 by 100 square and depot (50,50). Uniform locations come from seeds 11 and 29. The original instances use constant demand per tier. The heterogeneous generator samples varied integer demands and reduces demands inside each cyclic certificate group until capacity fits. That guarantees the certificate, but is a constructed family rather than every possible CVRP instance.

$$\rho = \frac{\sum_i d_i}{KQ}, \qquad \eta = \frac{\max_i d_i}{Q}$$

Load ratio rho tells how full the fleet must be overall. Largest-demand ratio eta tells how large the single biggest delivery is relative to one vehicle. Neither ratio alone proves packability.

Cluster first, route second

A partition G is a set of customer groups. Let L*(S) be the shortest depot tour for group S. Full CVRP minimizes the sum of L*(S) over feasible partitions. Instead, Track A chooses groups by a simpler score F. Track B then chooses visiting orders for those fixed groups.

$$C^* = \min_{\mathcal{G}\ \text{feasible}} \sum_{S \in \mathcal{G}} L^*(S)$$

The best partition for F can differ from the best partition for travel. This is the decomposition approximation. Exact route solving after imperfect grouping does not undo it.

Yes: this project uses QUBOs

[R2] quantum.Sampler.draw; [R3] routing.encoding and energy; [P3] Lucas 2014 for background

QUBO stands for **Quadratic Unconstrained Binary Optimization**. Binary means every variable is 0 or 1. Quadratic means terms involve one variable or products of two. Unconstrained means the energy can be evaluated on every bitstring; desired constraints may appear as penalties or may be checked after sampling.

$$E(b) = c_0 + \sum_a h_a b_a + \sum_{a<b} J_{ab} b_a b_b, \qquad b_a \in \{0, 1\}$$

A linear coefficient h_a rewards selection if negative and discourages it if positive. A pair coefficient J_ab charges an extra cost when both choices are 1. Positive J is repulsive: it discourages the pair from being simultaneously selected. Constants shift every score equally and do not change the winner.

$$b_a^2 = b_a, \qquad Q_{aa} = h_a, \qquad Q_{ab} = Q_{ba} = J_{ab}/2$$

The second equation is the symmetric-matrix convention for E = c0 + b^T Q b. The factor of one half avoids counting pair interactions twice. The code instead stores a symmetric J and evaluates its strict upper triangle once. Its stored J is not a symmetric Q matrix to use directly in b^T J b.

What is implemented and what is only derived?

Implemented: a Track A quadratic subset energy and two Track B routing QUBOs. Derived for comparison: full assignment variables plus binary capacity slack. Installed QUBO Solver and QoolQit are not the execution engines of these custom experiments: the source manually forms coefficients and calls Pulser, SciPy, and QutipBackendV2.

Say it to the judges: QUBO is the mathematical representation; using the qubo-solver package is a separate software choice. Our implemented quantum pipeline constructs its own binary energies.

Turning rules into penalties: the textbook model

[R2] MATH_WRITEUP sections 4-6; model comparison, not the implemented Track A optimizer

For an equality g(b)=0, add A g(b)^2. The penalty is zero when the rule holds and positive otherwise. For customer coverage, missing the customer or assigning it twice both give a nonzero penalty.

$$A\left(\sum_k x_{ik} - 1\right)^2 = A\left(1 - \sum_k x_{ik} + 2\sum_{k<r} x_{ik}x_{ir}\right)$$

Capacity needs more care. (load-Q)^2 penalizes a perfectly legal half-empty vehicle. Introduce slack s_k = Q-load, the unused capacity, and enforce load+s_k=Q instead.

$$s_k = \sum_{\ell=0}^{B_Q-1} 2^\ell u_{k\ell}, \qquad B_Q = \lceil \log_2 (Q+1) \rceil$$

$$E_{\text{slack}} = F(x) + A\sum_i \left(\sum_k x_{ik} - 1\right)^2 + B\sum_k \left(\sum_i d_i x_{ik} + s_k - Q\right)^2$$

Each slack bit consumes an atom. Extra binary-representable slack values above Q cannot make the residual zero for a nonnegative load. Squaring gives only linear and pair terms, because every variable is binary.

If a validated certificate has clustering score U, A=B=U+1 is a sufficient conservative bound for this algebraic model: F is nonnegative, any integer equality violation costs at least U+1, and the certificate with correct slack costs U. This proves feasibility of exact global minimizers of the target penalty model. It does not prove an atom array realizes its coefficients or that a finite pulse finds the minimum.

Say it to the judges: We derived the standard exact penalty model to count its cost. We did not claim our slack-free heuristic is an equivalent compressed version of it.

Atom accounting and the clustering score

[R2] BALANCE, pair_costs, clustering_cost; [R1] 80-atom challenge envelope

Representation	S	M	L
Direct customer / vehicle / position: N²K	32	300	900
Assignment: NK	8	30	60
Assignment plus binary slack	14	45	80
Implemented clustering: peak min(N,8)	4	8	8
Separate four-customer routing: edge / position	10 / 16	10 / 16	10 / 16

The sequential method reuses atoms across vehicle decisions. It reduces **peak** atom count, not necessarily total work. Count repeated preparations, backend calls, and samples as well. Clustering and routing registers are executed separately, so their peaks are not added as simultaneous atoms.

$$w_{ij} = \frac{c_{ij}}{100\sqrt{2}}, \qquad \lambda = 0.15$$

$$F(\mathcal{G}) = \sum_{S \in \mathcal{G}} \left[\lambda |S|^2 + \sum_{i<j,\ i,j \in S} w_{ij} \right]$$

Inside the official square, w is between 0 and 1. The squared-size term favors balanced sizes; the pair-distance term favors compact groups. For fixed total customers, moving one customer from a much larger group to a smaller one reduces the size term if the transfer is feasible.

Lambda is fixed before these experiments, not learned from exact references. The score has no direct depot-distance term and no visiting-order term. It is dimensionless; calling it kilometers or total route energy would be wrong. The planet demo uses 100 sqrt(3) because its application coordinates are three-dimensional.

Track A: the actual binary energy

[R2] quantum.Sampler.draw and run.evaluate

At one stage R is the remaining customer set, L is the number of vehicles still available, and W is a seeded window of at most eight remaining customers. A bit b_i selects customer i in W for the current vehicle. The remaining customers cannot be selected in this call.

$$\mu = 2\lambda\frac{|R|}{L}, \qquad \kappa_A = 4$$

$$E_A(b) = \kappa_A\left[\lambda\left(\sum_i b_i\right)^2 + \sum_{i<j} w_{ij}b_ib_j - \mu\sum_i b_i\right]$$

The negative reward term encourages selecting customers; the positive size and pair terms resist oversized or dispersed groups. Ignoring pair distances and allowing continuous group size m, lambda m²-mu m has derivative 2 lambda m-mu. Its minimum is m=|R|/L. That motivates mu; it is not a capacity guarantee.

$$h_i = \kappa_A(\lambda - \mu), \qquad J_{ij} = \kappa_A(2\lambda + w_{ij})$$

Expansion uses (sum b_i)² = sum b_i + 2 sum b_i b_j. Thus h is shared and every J is positive, matching two important native restrictions. These coefficients already include the physical scale; do not multiply by four again during mapping.

Each measurement produces a proposed subset. Reject the entire trajectory if selected demand exceeds Q or remaining demand exceeds (L-1)Q. Accepted customers are removed. The last vehicle receives the remainder if it fits. The current remaining-load test is necessary but not sufficient for heterogeneous packing. It does not call an exact packability solver online.

Say it to the judges: Capacity is enforced by explicit rejection and final validation. Our pulse energy favors plausible groups; it does not encode the complete capacity inequality exactly.

What an atom represents

[P1] Pulser conventions; [R2] quantum.py; [R3] routing.py

A neutral atom has no overall electric charge. The analog model uses two relevant states: a lower ground state |g> and a highly excited Rydberg state |r>. The latter has strong interactions with nearby excited atoms. In the chosen measurement basis, ground gives 0 and Rydberg gives 1.

$$n_a = |r\rangle\langle r|_a, \qquad X_a = |g\rangle\langle r|_a + |r\rangle\langle g|_a$$

n is the excitation-occupation operator: it returns 1 for the excited state and 0 for ground. X mixes the two states. A register is an ordered set of atoms and their coordinates. Bitstring order follows register order, which must then be mapped to customer IDs or routing variables.

Superposition and measurement

$$|\psi(T)\rangle = \sum_b \alpha_b |b\rangle, \qquad p(b) = |\alpha_b|^2$$

Before measurement, the state contains amplitudes for different bitstrings. It is not a readable list of all answers. Measurement samples one bitstring with the corresponding probability. Repeated shots build a histogram. Zero observed shots for a pattern does not prove its true probability is zero.

An occupation trace plots the expectation of n_a, the probability-like average excitation of atom a. It is not a definite delivery route at each time. In the routing demo, an atom can mean an edge or a customer-position pair rather than a customer itself.

What is simulated here

QutipBackendV2 evolves a full state vector numerically on a CPU. This avoids MPS bond truncation, but still uses numerical solver tolerances. Noise and hardware modulation response are disabled. The installed emu-sv and emu-mps packages are not the backends used for the saved official benchmark.

The Hamiltonian: the physics cost rule

[P1] Pulser Hamiltonian conventions; [R2] quantum.py; local Pulser 1.9.1 device values

$$\frac{H(t)}{\hbar} = \frac{\Omega(t)}{2}\sum_a X_a - \delta(t)\sum_a n_a + \sum_{a<b}\frac{\Gamma}{r_{ab}^6} n_a n_b$$

Term	Physical role	Optimization interpretation
Omega X / 2	Laser drives transitions	Mixes binary choices during evolution
-delta n	Energy shift for excitation	Shared linear coefficient h=-delta
Gamma n_a n_b / r^6	Repulsive Rydberg interaction	Positive pair coefficient J

Gamma is Pulser's interaction_coeff, approximately 865723.02 rad per microsecond times micrometer^6. It is C6 divided by hbar, not C6 in joules. Omega and delta are angular frequencies in rad per microsecond; register positions are in micrometers; pulse durations are entered in nanoseconds.

$$\Omega(T) = 0 \quad \Longrightarrow \quad \frac{E_{\text{phys}}(b)}{\hbar} = -\delta_f \sum_a b_a + \sum_{a<b}\frac{\Gamma}{r_{ab}^6} b_a b_b$$

At zero final drive the Hamiltonian is diagonal in the bitstring basis. Each string has a classical energy. This has the QUBO form. The desired mapping is h=-delta and J=Gamma/r^6, after applying the chosen energy scale. During the pulse, the X term is present, so evolution is not just evaluating a static cost table.

Say it to the judges: Energy is the physical score assigned to selections. We want useful solutions to occupy low-energy states, but only to the extent that the realized Hamiltonian represents our intended model.

Detuning, amplitude, phase, and units

[P1] sign convention checked; [R2-R3] fixed phase and pulse settings

Detuning delta describes the laser frequency offset in the rotating-frame convention used by Pulser. The energy term is -delta n. Positive detuning lowers the energy of an excitation; negative detuning raises it. A positive number called detuning is therefore not a positive penalty in this convention.

$$\Delta E_a/\hbar = -\delta_f \quad \text{(one isolated excitation)}$$

Example: delta=-12 makes one excitation cost +12 rad per microsecond before pair interactions. Delta=+1.8 makes it gain -1.8. If two atoms are excited, add their positive interaction too. Detuning rewards excitation globally, while interactions can make nearby pairs expensive.

Rabi frequency Omega controls the laser-driven mixing rate between |g> and |r>. It changes amplitudes during evolution, not the final diagonal pair penalties. The factor one half in the Hamiltonian matters. All implemented pulses here have phase zero, giving a real X drive; phase is not optimized.

$$P(r,t) = \sin^2(\Omega t/2), \qquad \Omega T = \pi \text{ (resonant pi pulse)}$$

The single-atom tutorial used T=1000 ns=1 microsecond and Omega=pi rad per microsecond, giving complete excitation ideally. Interacting multi-atom sweeps are more complicated; the one-atom formula does not predict their success rates.

Three quantities with different meanings

A 5.2-microsecond pulse is simulated physical duration. Roughly 13 seconds in the saved comparison is CPU wall time for a hybrid pipeline. A route length is a distance. None can be substituted for another. Multiplying a target energy by a positive constant preserves its mathematical minimizer, but can change dynamics under finite control bounds and a fixed pulse duration.

Embedding: fitting mathematics into geometry

[R2] quantum.embed; [P2] device restrictions checked against the installed device

$$J_{ab}^{\text{real}} = \Gamma / r_{ab}^{6}, \qquad r_{ab}^{\text{desired}} = (\Gamma / J_{ab}^{\text{target}})^{1/6}$$

Each desired coupling implies a distance, but all distances must belong to one shared planar layout. Arbitrary pairwise distances need not be geometrically consistent. Native couplings are positive and nonzero at finite separation. In routing, desired zero couplings still become unwanted physical interactions.

The embedder starts from a centered grid with spacing 9 micrometers. SLSQP, a constrained numerical optimizer, fits coordinates to squared coupling residuals. It uses at most 120 iterations and ftol=10^-7. It accepts a legal improving layout; otherwise it keeps the initial grid. This is local optimization, not a proof of the best possible layout.

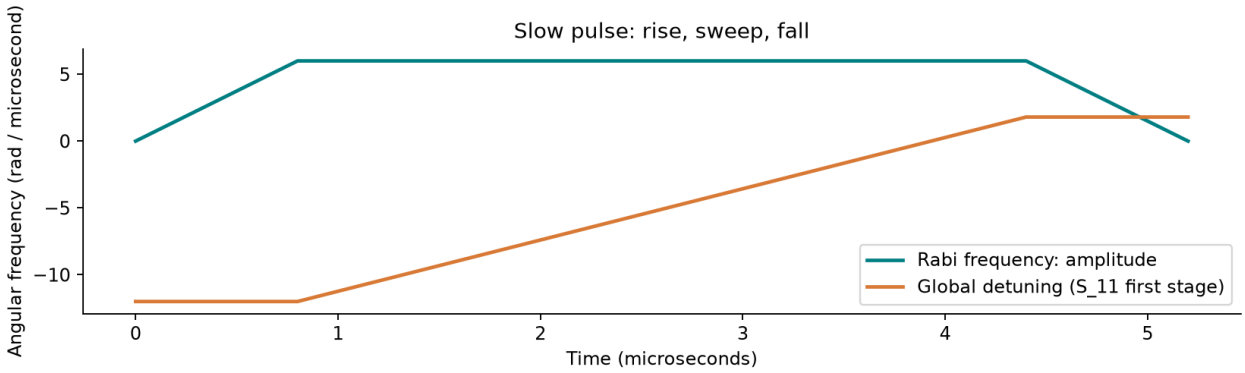
$$\varepsilon_J = \frac{\sqrt{\sum_{a<b} (J_{ab}^{\text{real}} - J_{ab}^{\text{target}})^2}}{\sqrt{\sum_{a<b} (J_{ab}^{\text{target}})^2}}$$

Device restriction	Envelope	Implementation
Atoms	At most 80	Clustering at most 8; routing 10 or 16
Separation / radius	At least 5 / at most 38 micrometers	Fit uses 5.001 / 37.999 margins
Rabi amplitude	At most 12.566 rad / microsecond	Uses 6
Absolute global detuning	At most 125.664 rad / microsecond	Checked by Pulser
Sequence duration / clock	At most 6000 ns / 4 ns	2000 or 5200 ns

The legal registers and pulse bounds are checked by Pulser, but these runs disable mimic_qpu and do not establish calibrated trap-layout availability. Small coefficient error also does not guarantee the same minimizing bitstring; near-tied states can change order.

Pulse shaping and the adiabatic idea

[R2] PULSES and Sampler.draw; [R3] simulate



Start with negative detuning and zero amplitude: every excitation costs energy, so the all-ground state is an easy initial low-energy state. Raise the amplitude, sweep detuning toward the target value, then lower the amplitude to zero. The plotted example uses the S_11 first-stage final detuning of +1.8.

Schedule	Rise	Sweep	Fall	Total
Fast clustering	400 ns	1200 ns	400 ns	2000 ns
Slow clustering / routing	800 ns	3600 ns	800 ns	5200 ns

$$i\,\partial_t|\psi(t)\rangle = (H(t)/\hbar)|\psi(t)\rangle$$

The emulator solves this evolution equation. The adiabatic idea is that sufficiently slow change can keep the state near a low-energy eigenstate if the relevant energy gaps and conditions permit it. We do not calculate a rigorous adiabatic guarantee. Finite-time errors can occur without noise, and a slower sweep cannot correct a wrong final Hamiltonian.

The spectral gap means a difference between eigenstate energies during evolution. It is unrelated to the percentage model gap or route gap used for scoring. The word gap therefore needs context.

Say it to the judges: We compare two schedules empirically. We do not claim that the slow schedule must be adiabatic or must outperform every alternative.

DMM: matching nonuniform linear costs

[R3] routing.simulate; [R2] MATH_WRITEUP section 10

Track A was chosen to have one shared linear coefficient, so global detuning can represent it. Routing QUBOs usually have different h values because different edges or endpoint customers have different travel costs. One global delta cannot represent them all.

$$h_a^{\text{real}} = -\delta_g - \varepsilon_a \delta_D, \qquad 0 \le \varepsilon_a \le 1, \quad \delta_D \le 0$$

DMM means detuning map modulator. In this project it is an explicitly added emulator control: fixed spatial weights epsilon multiply a shared nonpositive DMM waveform. It can raise selected excitation costs relative to the common baseline.

$$h_{\min} = \min_a h_a, \quad D = \max_a h_a - h_{\min}$$

$$\delta_g = -h_{\min}, \quad \delta_D = -D, \quad \varepsilon_a = (h_a - h_{\min})/D$$

Substitution gives -delta_g-epsilon_a delta_D = h_a. For D=0 use zero DMM and zero weights. Example: scaled targets -6.0 and -5.8 use global delta=6.0, DMM delta=-0.2, and weights 0 and 1; the resulting coefficients are exactly -6.0 and -5.8.

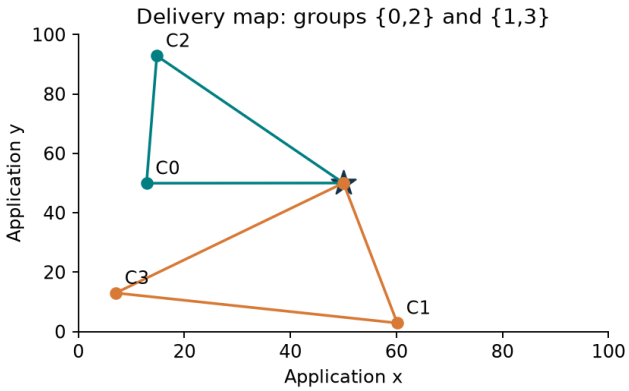
In global-only routing the code uses the mean of h, the least-squares common projection. This loses variable-specific travel information. In DMM mode the intended linear terms match at the end, subject to declared bounds; pairwise embedding mismatch remains.

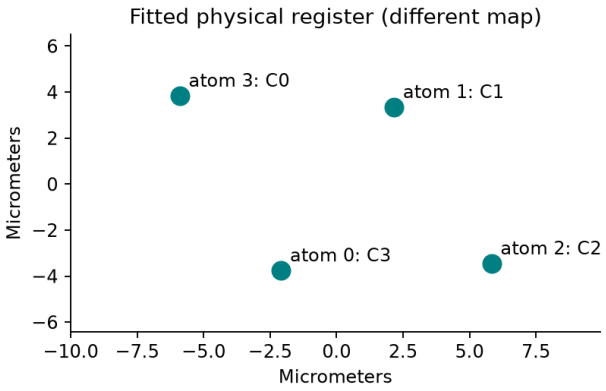
The routing DMM ramps from zero concurrently with the global sweep. Its custom channel specifies a 4 ns clock, 6000 ns maximum pulse, bottom detuning -125.663706, and total bottom detuning -10000. Those are declared emulation limits, not a hardware calibration or a promise that the competition QPU offers this control.

Say it to the judges: DMM and global-only results must remain separate. Better linear matching does not make the overall physical model exact.

Worked example: four real customer records

[R2] S_11 instance and independent recomputation for this guide





Customer	Coordinates (rounded)	Demand
0	(12.857, 49.928)	3
1	(60.150, 2.869)	3
2	(14.793, 92.821)	3
3	(7.042, 12.977)	3

There are two vehicles, each capacity 7; depot (50,50). Every customer needs 3. A vehicle can carry two customers (6), but not three (9). Every complete feasible solution therefore has two pairs. There are three unlabeled partitions; exchanging vehicle names does not create a new solution.

Partition	Grouping score F	Exact-routed distance
[[0, 1], [2, 3]]	2.238994	344.437415
[[0, 2], [1, 3]]	1.885881	294.499685
[[0, 3], [1, 2]]	2.176838	335.647268

The winner here is {0,2}/{1,3}; score 1.885881 and distance 294.499685. Exact reference computation and the independent S-tier CPLEX checks agree. This agreement between best F and best travel is instance-specific, not guaranteed by the decomposition.

Worked example: from a pair to its atom energy

[R2] saved S_11 fast bank; arithmetic recomputed from its pool and coordinates

The first stage has |R|=4, L=2, lambda=0.15. Therefore mu=0.6, h=-1.8 for each atom, and final delta=+1.8 rad per microsecond. J_ij=1.2+4w_ij. The actual saved register order is **[3, 1, 2, 0]**, so bitstrings must be interpreted in that order, not sorted customer order.

$$F(\{0,2\},\{1,3\}) = 0.15(2^2 + 2^2) + 0.303610 + 0.382271$$

$$F = 1.2 + 0.685881 = 1.885881$$

Consider selecting customers 0 and 2 for the first van. Its ideal pair coefficient is J_02=1.2+4(0.303610)=2.414441. Its two linear terms sum to -3.6. The ideal subset energy is therefore -1.185559 rad per microsecond. This is not the complete-partition F score: it is one vehicle-stage energy with a selection reward.

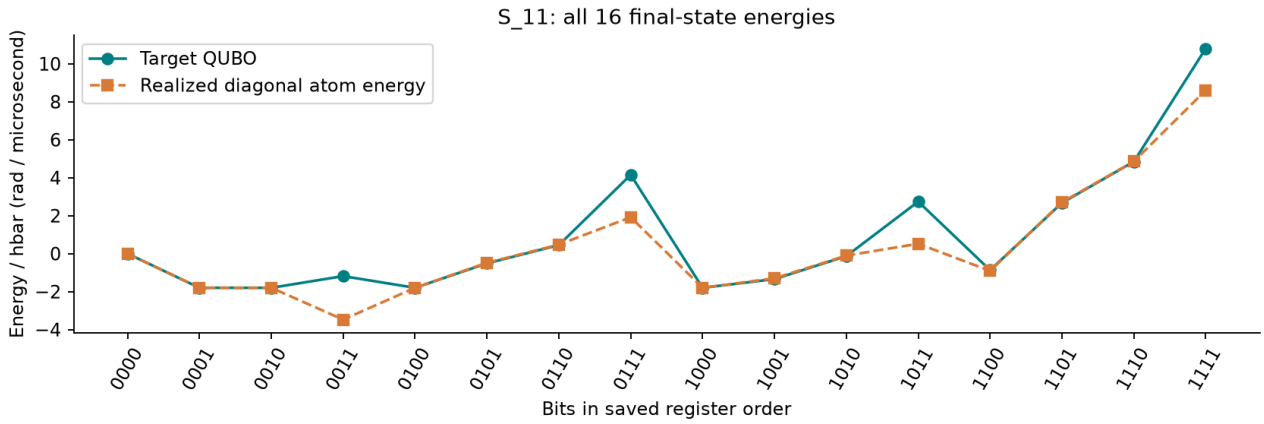
Bits	Selected IDs	Target E/hbar	Real E/hbar	Saved shots
1001	[3, 0]	-1.34202	-1.29982	113
0011	[2, 0]	-1.18556	-3.47523	193
1100	[3, 1]	-0.87092	-0.90903	70
0101	[1, 0]	-0.51296	-0.48070	48
1010	[3, 2]	-0.13106	-0.10502	64
0110	[1, 2]	0.44938	0.47373	40

These six pairs are valid stage choices because each leaves two customers for the other van. Their whole-partition scores need not follow the same order as their first-stage energies. The {0,3} pair is very close locally, but leaves the expensive {1,2} pair for the last van. This explains why minimizing one subset energy can be globally shortsighted.

Saved shot counts are finite simulated measurements from the colleague's bank, not values inferred from energy alone. Lower energy does not mean a deterministic measured outcome. The saved relative coupling error for this register is about 0.3052.

Worked example: the complete energy landscape

[R2] S_11 saved bank; original explanatory figure generated for this guide



Four bits give 2^4=16 possible selections. The green line evaluates the intended target energy; the orange line evaluates the actual final diagonal Hamiltonian from the saved atom distances. The mismatch is embedding error made visible. Lines only connect categories for readability; adjacent labels are not physical trajectories.

Some low-energy strings are not valid full-plan decisions. Feasibility is assessed by decoding customer IDs and checking loads, not by trusting the energy. At this stage, exactly two selected customers are needed. The energy does not explicitly impose that equality or the demand constraint.

Enumerating all 16 strings gives target minimum **0001**, selecting [0], and realized diagonal minimum **0011**, selecting [2, 0]. This is an energy-table calculation, not a claim that the pulse always measures either minimum.

The saved fast benchmark had 18 feasible full trajectories out of 32 (56.25%) and reached the model optimum in 7 of 32 (21.875%). Its best feasible model and route gaps were both zero. A later local eight-trajectory smoke run had four feasible trajectories and also found a zero-gap plan; the different sample budget is not the official benchmark.

Say it to the judges: The energy is a mechanism for proposing choices. The final competition score is calculated on the decoded, validated delivery plan, using a separate reference.

Track B: the route problem and position QUBO

[R3] routing.encoding(kind="position") and strict decoder

TSP means Traveling Salesperson Problem. A fixed cluster of n customers must be visited exactly once, starting and finishing at the depot. The depot stays fixed outside the customer positions. Use x_it=1 if customer i occupies position t, where t runs from 0 to n-1.

$$E_{\text{pos}} = T(x) + A\sum_i(\sum_t x_{it} - 1)^2 + A\sum_t(\sum_i x_{it} - 1)^2$$

$$T(x) = \sum_i \tilde{c}_{0i}x_{i0} + \sum_i \tilde{c}_{i0}x_{i,n-1} + \sum_{t=0}^{n-2}\sum_{i\neq j}\tilde{c}_{ij}x_{it}x_{j,t+1}$$

Read x as a table: rows are customers, columns are visiting positions. A valid table has exactly one 1 in each row and column. The row penalties stop repeated or missing customers; the column penalties stop two customers sharing a stop or leaving a stop empty.

$$h_{it} = -2A + \tilde{c}_{0i}\mathbf{1}_{t=0} + \tilde{c}_{i0}\mathbf{1}_{t=n-1}, \qquad c_0 = 2nA$$

Each row and column contributes -A to a variable, producing -2A. Pairs sharing a customer or position receive 2A; consecutive-position travel adds its normalized distance to the appropriate pair. n² variables give 16 atoms at n=4.

For a legal permutation all penalties vanish and target energy equals normalized route length. The decoder independently requires exactly n selected variables, n distinct customers, and n distinct positions. It rejects all other measured strings even if their physical energy is low.

Say it to the judges: The position QUBO is algebraically valid. Its many atoms, unwanted couplings, and embedding distortion can still make valid measured routes rare.

Track B: the smaller edge QUBO

[R3] routing.encoding(kind="edge"); [R2] latest clarified scope in MATH_WRITEUP

Here one bit e_ij means using the undirected connection between locations i and j. Include the depot, so n customers give n+1 vertices. A tour has exactly two incident edges at each vertex: arrive once and leave once.

$$E_{\text{edge}} = \sum_{i<j} \tilde{c}_{ij} e_{ij} + A \sum_{v=0}^{n} \left(\sum_{e \ni v} e - 2 \right)^2$$

$$\left(\sum_{e} e - 2\right)^2 = 4 - 3\sum_{e} e + 2 \sum_{e<f} ef$$

Each edge appears at two endpoint vertices. Consequently h_ij = normalized distance - 6A, edge pairs sharing a vertex get 2A, disjoint edge pairs get zero target coupling, and the constant is 4(n+1)A. There are n(n+1)/2 variables: 10 for four customers, compared with 16 in position encoding.

Why is there no subtour penalty here?

Degree two normally permits disconnected cycles. At four customers plus the depot there are only five vertices. Two disjoint simple cycles would require at least six vertices, so a degree-two graph must be one cycle. This reasoning applies to 2-4 customers, not every n. At five customers, two disjoint triangles are a counterexample. A one-customer depot trip needs the same undirected edge twice and is not represented by a binary simple cycle.

The code rejects edge encoding beyond four customers and the pipeline only quantum-routes nontrivial four-customer groups. Its decoder checks degree and connected traversal rather than trusting the penalty.

Say it to the judges: Our edge encoding saves atoms for the executed small routing cases. We do not claim degree-two penalties solve unrestricted TSP.

Penalties, scaling, and exhaustive algebra checks

[R3] PENALTY=10, SCALE=0.1; test_routing.test_both_qubo_energies_and_exact_decoders

Normalized travel coefficients lie between 0 and 1 for the stated geometry. A four-customer tour has five edges and costs at most five. A nonzero squared integer constraint violation costs at least A=10. With nonnegative travel contributions, every invalid state therefore lies above some valid tour in the exact target energy.

$$A_{\mathrm{TSP}} = 10 > 5, \qquad E_{\mathrm{scaled}} = 0.1E_{\mathrm{target}}$$

The penalty is fixed before examining solutions. The bound is conservative. Multiplying all coefficients and the constant by 0.1 preserves target minimizers. Physical embedding, projection of h, and finite pulse evolution may still change which state is favored or observed.

Check for n=4	Edge	Position
Binary variables	10	16
All candidate bitstrings	2^10 = 1024	2^16 = 65536
Strictly valid strings	12	24
Distinct symmetric tours	12	12
Reason for extra position states	One orientation per undirected edge set	Two reverse customer orderings per tour

The test exhaustively evaluates all candidates for a fixture. It checks accepted energies against normalized route lengths and verifies that invalid target states cannot beat the valid optimum. This is stronger than checking a few examples, but it isolates algebraic correctness. It does not prove a physical embedding has the same minimum or that a simulator will sample it with high probability.

A four-customer random valid tour already has a 1/12 chance of being the unique optimum up to reversal in a generic symmetric fixture. Drawing 2000 valid random permutations is consequently a strong small-cluster baseline. It also receives a feasibility advantage over raw atom bitstrings, which must be stated rather than concealed.

How Track B assembles the full answer

[R3] route_pipeline in track_b.py; saved 12-instance benchmark

Group size	Routing action	Included in quantum routing gap?
0-3	Exact classical trivial route	No
4	Edge/global, edge/DMM, position/DMM, or random comparison	Quantum variants only
More than 4	Declared exact classical size fallback	No

Track B fixes the best feasible model-selected clustering. Different route encodings then use the same groups, so the comparison is about routing rather than changing assignments underneath it. For each eligible quantum group it takes 2000 shots, decodes strictly, and returns the shortest valid measured tour.

If a four-customer quantum batch has no valid route, that method's end-to-end answer is unavailable. The method does not invoke classical repair after observing this failure. Trivial and oversized groups follow the declared classical policy in advance.

The official benchmark uses precomputed exact subset routes for scoring and its explicitly classical trivial/fallback groups. These are substantial classical contributions. The standalone timing comparator instead computes its classical fallback work inside the timed function; neither should be described as an all-quantum solver.

Quantum coverage is successful nontrivial quantum-routed groups divided by all nontrivial groups, including size fallbacks. A run with only tiny trivial groups is not evidence of useful quantum routing. Failure gaps are blank, not zero.

The saved benchmark has 12 instances: S/M/L × two seeds × uniform/heterogeneous. Five pipeline methods give 60 rows. The saved summary reports 49 feasible returned pipelines. Edge/global and edge/DMM each return 11 of 12; position/DMM returns 5 of 12. These counts include shared clustering failures and trivial-only instances, so they are not raw shot success probabilities.

Exact references: where the answer key comes from

[R2] core.references and cplex_check; [R3] independent tests

The code enumerates capacity-feasible customer subsets and computes a shortest depot tour for each. Held-Karp dynamic programming avoids trying every ordering independently. Let D(S,j) be the shortest depot-starting path visiting exactly S and ending at j.

$$D(\{j\}, j) = c_{0j}, \qquad D(S, j) = \min_{i \in S \setminus \{j\}} [D(S \setminus \{j\}, i) + c_{ij}]$$

$$L^*(S) = \min_{j \in S} [D(S, j) + c_{j0}]$$

A second dynamic program chooses a partition of all customers into at most K feasible subsets. It anchors the smallest remaining customer, so exchanging vehicle labels does not count the same partition repeatedly. Using F(S) as subset cost gives the clustering optimum; using L*(S) gives the CVRP optimum.

$$V(R, L) = \min_{S \subseteq R,\ \min R \in S} \{q(S) + V(R \setminus S, L - 1)\}$$

The recurrence also allows fewer than K nonempty groups and counts optimal unlabeled partitions. Base cases accept an empty remaining set and reject nonempty sets with no vehicles left. Feasible subset generation handles capacity.

For S, an independent enumeration of labeled assignments and direct CPLEX set-partition solves cross-check both optima. CPLEX uses a 30-second limit, one thread, and zero relative MIP gap; statuses are saved. The tested S fixtures agree. Exact reference work is exponential and can dominate evaluation time; small instance dimensions make it manageable here.

A nuance judges may notice

run.evaluate selects min(model_cost, routed_cost), so exact routed cost is used as a tie-break between equal model scores. It does not construct or repair a sampled grouping, but it is an oracle-assisted tie-break in the benchmark selection. State it if claiming strict separation of evaluation and optimizer; choose a reference-free tie-break in a future controlled comparison.

Every score and gap, with the correct denominator

[R1] evaluation; [R3] track_b.route_pipeline; [R2] run.evaluate

$$\mathrm{gap}(C;B) = 100\,\frac{C-B}{B}$$

A gap compares a cost with a positive reference B. Zero matches the reference; 10% means cost is 1.1 times the reference. No feasible solution means undefined, not zero. If B=0 a separate absolute-difference policy is needed; the current direct divisions assume positive fixture references.

Metric	Compared quantities
Model gap	Selected F versus exact constrained F optimum
CVRP gap	Exact-routed selected groups versus full CVRP optimum
Routing gap	Quantum routes versus exact routes on the same eligible groups
End-to-end gap	Actual merged routes versus full CVRP optimum
Decomposition gap	Exact-routed F-optimal groups versus full CVRP optimum

Let C_ref be the exact CVRP optimum, C_model the exact-routed model-optimal partition, C_shot the exact-routed selected partition, and C_end the actual final tours.

$$C_{\mathrm{end}} - C_{\mathrm{ref}} = (C_{\mathrm{model}} - C_{\mathrm{ref}}) + (C_{\mathrm{shot}} - C_{\mathrm{model}}) + (C_{\mathrm{end}} - C_{\mathrm{shot}})$$

The three terms are decomposition loss, clustering loss, and routing loss. They sum in original distance units; percentages with different denominators cannot simply be added. Clustering loss can be negative because a nonoptimal F score may give a shorter route. Track B asserts this distance identity for one consistent selected clustering.

Historical Track A separately selects the best-F shot and the best exact-routed shot, so its model and CVRP gaps can refer to different trajectories. It also saves the model-selected route gap to expose this. Track B consistently uses its fixed best-model grouping for its decomposition identity.

Probabilities: what a shot and a trajectory mean

[R2] conditional banks and evaluate; [R3] routing histograms and summary semantics

$$\widehat{P}_{\text{feasible}} = \frac{N_{\text{feasible trajectories}}}{N_{\text{attempted trajectories}}}$$

$$\widehat{p}_{\text{valid}} = \frac{\sum_{b\ \text{valid}} N_b}{N_{\text{shots}}}, \qquad \widehat{p}_{\text{hit}} = \frac{\sum_{b\ \text{valid, hit}} N_b}{N_{\text{shots}}}$$

A shot measures one register. A trajectory assembles several vehicle-stage choices into one full assignment. A backend call evolves one conditional sequence. A bank is a histogram of 1000 measured clustering strings for a particular window and detuning. Cached banks are reused when that condition recurs.

Official clustering experiments sample 32 complete trajectories from these finite empirical distributions. The demos use 64. Bank reuse means trajectories share estimation uncertainty; they are not independent new physical experiments. Routing uses 2000 raw bitstring samples per eligible group. Multiple counts of the same route remain distinct shots, not distinct tours.

Reference-hit tolerance is 10^-6 percent, corresponding to a ratio tolerance of 10^-8. Failures remain in denominators. Stage acceptance rate, full-trajectory feasibility, best-of-batch success, and per-shot hit probability answer different questions.

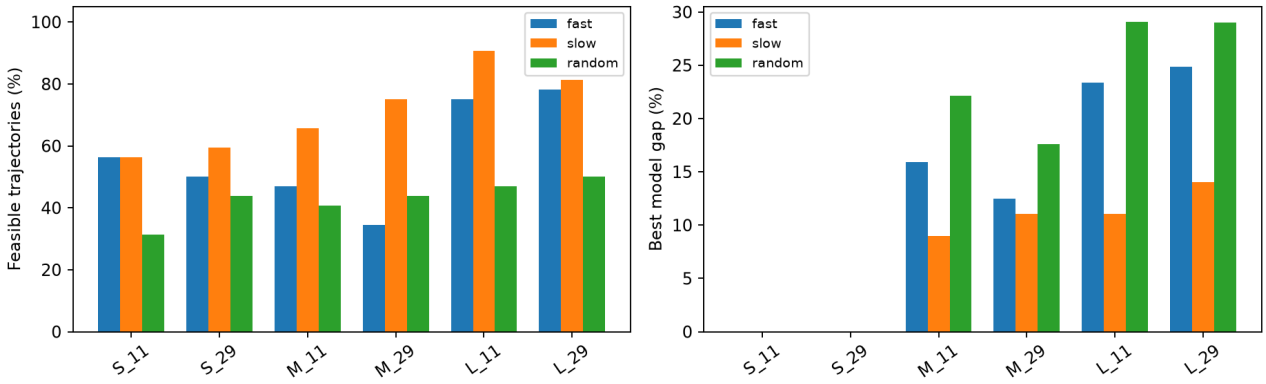
$$P(\text{at least one hit in } M) = 1 - (1 - p)^M$$

This formula assumes independent identically distributed trials with fixed p. It cannot automatically be applied to adaptive multi-stage selection and shared estimated banks. Products of route hit rates are conditional on fixed selected groups; they are not unconditional probabilities of solving full CVRP optimally.

Say it to the judges: We report empirical rates with their sample budgets. We do not infer statistical significance, quantum advantage, or exact true probabilities from these small cached-bank experiments.

Reading the actual results and timing

[R2] historical Track A CSV; [R3] Track B CSV and US timing.json



Left: full-trajectory feasibility, higher is better. Right: best clustering-model gap, lower is better. Blue is fast, orange slow, green random. These are saved historical Track A runs with equal demands. Slow generally improves the grouping metric here, but these bars are not a plot of driving distance or a claim of statistical significance.

For M_11 the best exact-routed CVRP gap is 20.52% fast, 21.95% slow, and 15.27% random. Thus a better grouping score did not guarantee a better delivery route. Large-tier coupling errors remain substantial, roughly 0.61-0.68 on average across the quantum M/L rows.

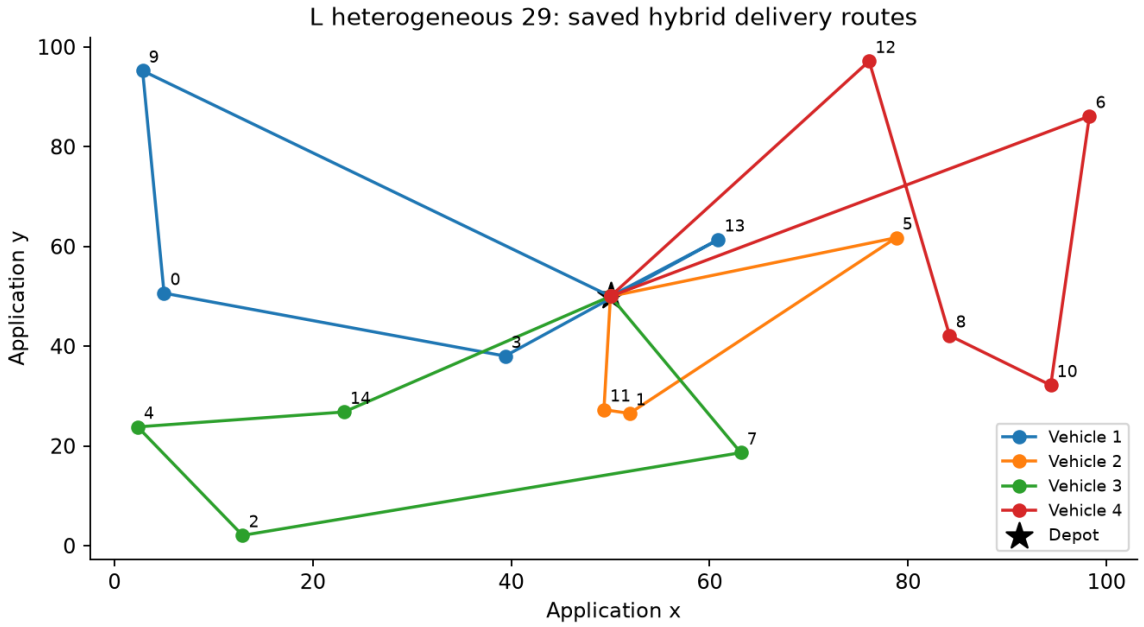
Saved timing, three repeats	Median seconds	Route excess
Exact classical CVRP	0.28835	0%
Quantum-assisted CPU emulator	12.89708	39.282%

The ratio is 44.73: the CPU-emulated pipeline is slower in this fixture. Measurements were saved on macOS arm64 with Python 3.11.15. Timings exclude imports, input loading, export, and offline reference scoring; fresh solver caches are used. They include embedding and simulation. They are not this Windows laptop's benchmark and not real-QPU timing.

The classical comparator proves an optimum while the quantum-assisted method is heuristic and not equally accurate. Report runtime and quality together. No real hardware speedup is established.

The route plots and the two visual demonstrations

[R3] report_b and saved route plot; [R4] us_demo, space_demo, Atlas browser files



The star is the depot; numbered dots are customers; each color is a vehicle. Connecting segments show the visiting order, including the return trip. A valid picture does not prove optimality. The image title distinguishes measured edge/DMM tours and classical contributions. Distances are straight-line application costs, not roads.

Earth replay: fifteen illustrative US city centers with projected straight-line distances. It replays saved assignment choices, checks, routing occupations, and final routes. Kilometer labels come from the projection's scale, not a road network.

Atlas Orbit: ten fictional planets with fixed XYZ positions and a station. Actual 3D Euclidean distances enter the score. One four-planet group is quantum-routed and two trivial groups are classical. Space-distance units are fictional; no gravity, orbit mechanics, fuel, or time-dependent positions are modeled.

The browser reads exported JSON. Playback does not evolve a quantum state or submit jobs. Its route-candidate scan is the decoder's order over distinct valid measured patterns, not a chronological hardware shot movie. Occupation curves are quantum expectations. The physical atom register remains two-dimensional even when the application map is three-dimensional.

The software and the work completed so far

[R2-R4] source map; local environment inspection and latest Git history

File / component	Role
core.py	Generate and validate instances; F; Held-Karp and partition references; CPLEX checks
quantum.py	Subset proposals, atom embedding, global pulse, cached banks
run.py	Trajectory checks, selection, historical Track A metrics
routing.py	Two route QUBOs, strict decoders, DMM/global simulations
track_b.py	Combine fixed clustering and route variants; gaps and losses
report*.py, summarize_b.py	Plots, tables, reports from saved evidence
us_demo.py, space_demo.py	Record visual examples and export browser payloads
time_comparison.py	Measure classical and emulator wall time with fresh caches
MATH_WRITEUP.md / build_math_pdf.py	Latest comprehensive mathematics and typeset PDF build

Locally we cloned the team repository, built a Python 3.13 virtual environment, installed its pinned core stack plus notebook support, verified tutorial simulations and CPLEX, opened the folder in VS Code, and installed the Codex extension. GitHub tracks shared code; VS Code edits the local copy; .venv isolates Python dependencies.

The newest pulled commits add the comprehensive math source (69521cf) and its reproducible typeset PDF build (d7027f7). Earlier updates added Track B, the US replay, timing, and the 3D planet example. Their existing PDF remains in output/pdf inside the repository. This guide adds beginner explanations, worked energy arithmetic, score interpretation, and judging preparation.

This report does not change the optimization algorithm, upload anything, or rerun the complete recorded experiment set. The current source snapshot and current saved results are distinct evidence layers; historical Track A results retain their earlier capacity-check provenance.

What is proved, tested, measured, and still limited

Audit notes for honest defense; [R2-R4] tests and sources

Evidence level	What it supports	What it does not support
Algebraic derivation	Exact target-QUBO penalties under stated bounds	Faithful physical embedding
11 passing tests	Fixture algebra, references, geometry and saved trace consistency	All inputs, all regressions or statistical significance
Saved shot histograms	Finite empirical samples and decoded outcomes	Exact underlying probabilities or hardware execution
Exact reference DP	Optimum for the enumerated finite instance model	Real-world driving optimality
Timing repeats	CPU wall time for this fixture and machine	Real-QPU speedup or asymptotic advantage

Limitations to say before a judge has to ask

Sequential decisions, seeded windows, and a surrogate objective restrict search. Capacity is postselected. Heterogeneous load bounds do not prove future packability. Coupling errors can be large. Global-only routing loses nonuniform linear costs. DMM is emulator-only. Exact state-vector cost grows exponentially. Most small groups are classically routed, while larger ones can fall back. Position encoding sometimes produces no valid routes.

Full benchmarks use exact reference-derived route tables for declared classical evaluation and fallbacks, and best-F selection uses exact routed cost to break ties. These classical roles should be disclosed. Cached banks and best-of-shot selection also have costs. Noisy hardware, calibrated layouts, and modulation response are not simulated.

Improvement sequence

First isolate experiment output directories and expand validation regression tests. Then evaluate more independent seeds with uncertainty estimates and fair resource budgets. Measure native versus target ground-state rankings. Compare window sizes, grouping objectives including depot effects, and embedding strategies one at a time. Generalize routing only with justified subtour handling and a declared simulation budget. Keep failed cases in the tables.

Glossary: optimization and software

Use these definitions consistently in the team presentation

Term	Meaning here
CVRP	Choose customer groups and depot tours under vehicle capacity limits.
TSP	Find visiting order for one fixed customer group.
Instance / tier / seed	A dataset / its required size class / a reproducibility number.
Demand / capacity / load	Customer amount / vehicle limit / sum actually assigned.
Certificate / validator	Known feasible assignment / code checking legality.
Objective / cost function	The number the mathematical optimizer tries to minimize.
Surrogate	A convenient substitute objective; F is not route distance.
Constraint / penalty	A legality rule / an energy charge encouraging that rule.
QUBO	Quadratic unconstrained binary optimization energy.
Linear / quadratic coefficient	Effect of selecting one bit / selecting a pair.
Slack variable	Unused capacity encoded to turn an inequality into equality.
Encoding / decoding	Represent a decision in bits / interpret bits as a candidate.
Heuristic / exact solver	Method without optimality guarantee / method proving the finite optimum.
Dynamic programming	Cache optimal subproblem answers and combine them.
CPLEX	Classical mathematical optimization software used for S cross-checks.
Baseline / ablation	Comparison method / controlled removal or change of one component.
Reference / optimality gap	Answer key / percentage excess above it.
Canonicalization	Treat identical partitions with permuted vehicle labels as one.
Repository / branch / commit	Versioned shared files / line of development / exact snapshot.
Virtual environment	Project-specific Python interpreter and installed packages.

Glossary: physics and experimental statistics

[P1-P2] meanings checked against the executable model

Term	Meaning here
Pasqal / Pulser	Company / Python library for atom registers and laser sequences.
Neutral atom / Rydberg state	Uncharged atom / highly excited state with strong interactions.
Register / embedding	Ordered atom positions / fitting those positions to a target interaction model.
Hamiltonian H	Operator defining the simulated system's energy and time evolution.
Ground state / excited state	Lowest-energy state / a higher-energy state of a specified Hamiltonian.
Detuning delta	Laser-frequency offset; contributes -delta times excitation occupation.
Rabi frequency Omega	Laser-driven transition/mixing rate, entering as Omega X/2.
Phase	Drive orientation parameter; fixed at zero in these experiments.
Coupling J / blockade	Pair energy / suppression of nearby simultaneous excitation when interaction dominates.
DMM	Weighted nonpositive extra detuning control, emulation-only here.
Adiabatic / spectral gap	Slow-following idea / energy separation between instantaneous eigenstates.
Superposition / amplitude	Quantum combination of basis states / its complex coefficients.
Shot / histogram	One measured bitstring / counts across repeated measurements.
Occupation	Expectation of an atom being excited; not a definite route.
Trajectory / bank	Complete multi-stage assignment attempt / cached conditional shot histogram.
QPU / emulator	Physical quantum processor / classical numerical simulation.
State vector / MPS	Full amplitude representation / compressed tensor-network representation.
Bond dimension	MPS approximation-control size; not applicable to these state-vector runs.
Noise / modulation	Environmental errors / hardware response to programmed waveforms; disabled here.
Embedding error / sampling error	Coefficient mismatch / finite-data uncertainty; different error sources.

Judge questions: answer with the right scope

Suggested answers to rehearse, not memorized claims without understanding

Where is the quantum computation?

Pulser sequences evolve interacting atom models; measured subsets choose vehicle groups and measured routing bits choose tours for eligible four-customer clusters. All runs here are CPU emulations.

Did you use QUBO?

Yes: the sequential quadratic subset energy and two explicit TSP QUBOs. The full slack-based assignment model is a derived comparison, not our implemented capacity Hamiltonian.

What enforces capacity?

Measured subsets are rejected if selected or remaining demand bounds fail; complete assignments are validated. Heterogeneous remaining-load checks are necessary, not sufficient, for completion.

Why is detuning negative at the start?

Because the energy contribution is -delta n. Negative delta makes excitation expensive and gives an easy all-ground initial state at zero drive.

Why do atom positions matter?

The pair cost is Gamma/r^6. Distances program positive interactions, but all distances share one plane and cannot be set independently.

Why not just make the pulse slower?

Finite-time errors are only one issue. Slower evolution cannot fix the surrogate objective, capacity postselection, global-linear projection, or a distorted embedding.

Why is edge routing restricted?

Degree two can permit disconnected cycles. With five or fewer vertices two simple cycles cannot fit, but at six vertices two triangles can. We execute nontrivial routing only at four customers.

Have you shown quantum advantage?

No. The saved CPU timing fixture favors the exact classical solver in both runtime and route quality. Our contribution is a transparent constrained-hardware encoding experiment and its limitations.

Judge questions: scoring and evidence

Practical oral-defense checklist

Is energy the delivery distance?

Not in general. Track A energy favors a subset, F scores a whole grouping, and C measures travel. Track B target energies equal normalized route length only on legal states, up to constant/scale. Physical energies are approximate.

What is a zero gap?

It matches the chosen reference at the named objective level. Always specify model, CVRP, routing, or end-to-end gap. No feasible shot means undefined gap, not zero.

How do you know the reference is optimal?

Enumerate feasible subsets, compute exact tours using Held-Karp, and solve exact partition DP. S also cross-checks assignment enumeration and CPLEX. State the finite instance and cost model.

Are the 32 attempts independent quantum experiments?

They are full trajectories resampled from finite cached conditional banks. They share bank-estimation uncertainty. Routing uses 2000 raw samples per eligible cluster.

Does DMM make it exact?

It matches target linear coefficients under the declared emulation controls; pairwise geometry remains approximate. DMM is kept separate from global-only results.

What does the animation show?

Saved measured proposals and checks, plus a scan of decoded valid route patterns. It does not run a simulator in the browser, and occupation curves are expectations rather than routes.

What classical help is used?

Generation, embedding optimization, rejection, selection, validation, exact scoring and declared route fallbacks. Exact routed cost is also a tie-break for equal clustering-model scores in evaluate.

Where did AI help?

Development, derivations, debugging, figures, documentation, and this study guide. The team must review every claim and be able to derive and explain the actual equations. This guide is not a substitute for that review.

Source trail and how to reproduce checks

Snapshot d7027f7 | Prepared 24 October 2026 | Sources distinguish code, evidence, and background

[R1] Challenge specification. User-provided Pasqal-challenge-final DQC HACKATHON.pdf, version 3.0, dated 24 October 2026. Sections 2-10 define assumptions, tiers, hardware envelope, tracks, metrics, and deliverables.

[R2] Team model and Track A source. core.py, quantum.py, run.py, MODEL.md, MATH_WRITEUP.md, instances/S_11.json, results/S_11_fast.json, results/S_11_reference.json, results/results.csv. Paths are relative to the repository root.

[R3] Track B source and evidence. routing.py, track_b.py, TRACK_B_MODEL.md, track_b_results/results.csv, clusters.csv, summary.json; test_challenge.py, test_routing.py. Historical saved measurements were inspected, not all regenerated for this guide.

[R4] Demonstrations and timing. us_demo.py, time_comparison.py, us_demo_results/timing.json; space_demo.py, space_demo_results/dashboard.json and test_space.py; sites/atlas-route-lab/README.md and dist/. The browser uses exported evidence JSON.

Repository snapshot: GitHub source at the reviewed commit. Latest repository: ericmao2007/DuQuantum-Pasqal.

[P1] Pulser conventions. State, bit order and Hamiltonian sign conventions. This background page is an older documentation version; installed Pulser 1.9.1 source and sequence validation are the executable authority for this run.

[P2] Pasqal hardware documentation. Hardware specifications and channel constraints. Numerical values in this guide were checked against the local device; no calibration claim is inferred.

[P3] Andrew Lucas (2014). Ising formulations of many NP problems, Frontiers in Physics 2:5. Background for squared-constraint constructions; this project does not claim to invent QUBO encoding.

Safe checks from the project root in VS Code PowerShell

```
.\.venv\Scripts\python.exe -m unittest
challenge.test_challenge challenge.test_routing challenge.test_space -v
```

Type the unittest command on one line. Run .\.venv\Scripts\python.exe -m pip check for dependency consistency. For a full new Track B experiment use python -m challenge.track_b with the virtual environment interpreter. It can be expensive and replaces matching aggregate benchmark artifacts; archive outputs first. Report-building commands regenerate tracked report files.

Local working branch: setup/windows-environment. Windows Python 3.13 environment differs from the saved macOS/Python 3.11 timing machine. Preserve requirements.windows.lock.txt as the local environment record. No complete rerun or real-QPU execution was performed for this guide.

Appendix: expand the capacity QUBO yourself

[R2] MATH_WRITEUP section 5; algebraic comparison, not the implemented slack-free optimizer

Let a_l=2^l be a slack-bit weight. For one vehicle, expand B(load+slack-Q)^2 by ordinary multiplication, then use $x^2=x$ and $u^2=u$. The capacity contribution consists of the following terms. Each pair appears once in the canonical upper-triangle convention.

Term type	Coefficient before multiplying by B
Constant	Q^2
Customer x_i	$d_i^2 - 2Q\, d_i$
Customer pair x_i x_j	$2\, d_i\, d_j$
Slack bit u_l	$a_l^2 - 2Q\, a_l$
Slack pair u_l u_m	$2\, a_l\, a_m$
Customer / slack x_i u_l	$2\, d_i\, a_l$

Now add the clustering objective and coverage penalty. A customer-assignment bit x_ik has linear coefficient h_ik = lambda - A + B(d_i²-2Qd_i). Same-vehicle customer pairs have 2 lambda + w_ij + 2B d_i d_j. Copies of the same customer on two different vehicles couple with 2A. Slack linear coefficients remain B(a_l²-2Qa_l). The whole constant is AN+BKQ².

$$h_{ik} = \lambda - A + B(d_i^2 - 2Qd_i), \qquad c_0 = AN + BKQ^2$$

For Q=7, d=3, and slack weights 1,2,4: the customer capacity-linear term is -33B; slack linear terms are -13B, -24B, and -40B; customer-pair terms are 18B; customer/slack terms are 6B,12B,24B. These differ substantially across variable types. A single global detuning cannot independently reproduce them.

That is why "it fits in 80 atoms" is only the first check. Coefficient sizes, shared linear controls, dense desired interactions, planar geometry, and simulation expense all matter. The sequential implementation avoids this complete model by changing the decision process, not by proving these coefficients unnecessary in an equivalent encoding.

Appendix: Ising spins and blockade intuition

[R2] optional background; [P1] be explicit about state and Pauli conventions

QUBO uses bits 0 and 1. An Ising model commonly uses spins -1 and +1. Define an abstract spin s_a=2b_a-1, so b_a=(1+s_a)/2. This is an algebraic change of variables; it is not a statement that Pulser's matrix-basis Pauli Z has this sign.

$$E(b) = c_0 + \sum_a h_a b_a + \sum_{a<b} J_{ab} b_a b_b$$

$$E(s) = c_0' + \sum_a H_a s_a + \sum_{a<b} K_{ab} s_a s_b$$

$$K_{ab} = J_{ab}/4, \qquad H_a = h_a/2 + \tfrac{1}{4} \sum_{b \neq a} J_{ab}$$

The constant also changes, but not which state minimizes the energy. This explains why QUBO and Ising formulations are closely connected and why the neutral-atom occupation Hamiltonian can represent binary optimization. Always state the spin convention before comparing coefficients between papers or software.

Rydberg blockade

When the interaction energy between two excited atoms is large compared with the drive scale, simultaneous excitation can be suppressed. A useful heuristic blockade radius is the separation where Gamma/r^6 is comparable to Omega. It is a scale, not a rule deleting interactions outside a cutoff.

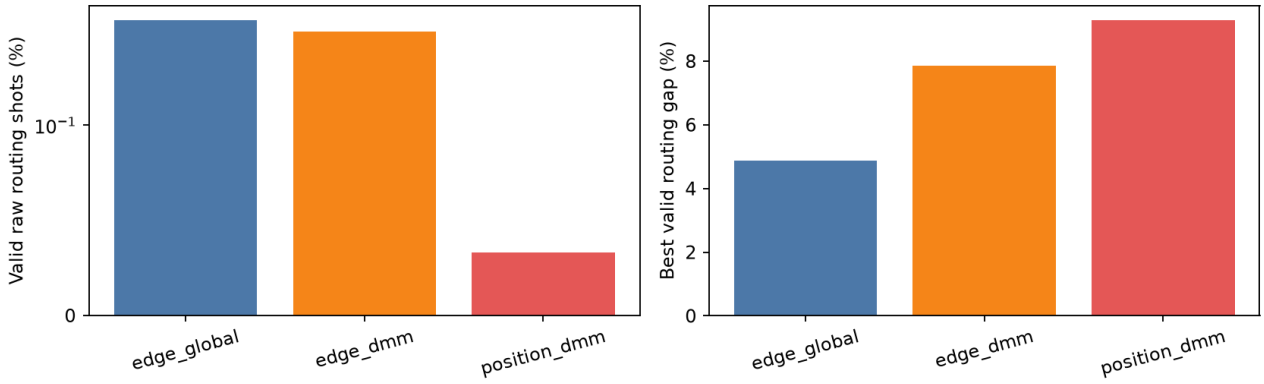
$$R_b = (\Gamma/\Omega)^{1/6}$$

At the programmed Omega=6 rad per microsecond, this estimate is 7.242 micrometers. The actual simulator retains interactions between every pair, including long-range tails. Pulse detuning, geometry, and many-body dynamics also influence excitation; blockade alone does not determine a route.

Energy minimization is not the atoms using less delivery fuel. We encode desirable binary decisions in low-energy configurations of a controlled quantum model, then interpret measurements as delivery decisions. The physical energy landscape is a computational representation of a chosen cost function.

Appendix: read the routing comparison carefully

[R3] report_b.build and track_b_results/clusters.csv; saved colleague measurements



Blue is edge/global, orange edge/DMM, and red position/DMM. The left plot averages the percentage of raw shots that decode to legal tours across eligible cluster records. Higher is better. Its vertical scale is **symlog**, not an ordinary linear percent scale; read labels rather than judging bar-height ratios.

The right plot averages each cluster's best valid route gap. Lower is better. The plotting function excludes blank gaps, so a batch producing no valid tour is absent from this particular quality average. A failed batch is still a failure in end-to-end metrics, and its zero feasibility remains in the left-hand average. This difference is essential when comparing the red bar with the others.

Variant	Batches	Mean valid shots	Batches with gap	Mean best gap
edge_global	15	0.4100%	15	4.871%
edge_dmm	15	0.3500%	15	7.843%
position_dmm	15	0.0333%	8	9.286%

The table is recomputed directly from saved cluster records. It reports an unweighted average of per-cluster percentages, matching the plot; it is not an average over every pipeline row. Only four-customer eligible groups enter these quantum-routing variants.

In these saved runs, edge models produce more valid measured tours than position encoding. DMM does not uniformly improve route quality despite matching linear terms: physical pair distortion and finite evolution still matter. These comparisons are empirical and limited, not a theorem that one encoding or control always wins.